**SOFTWARE ENGINEERING LAB**

*Software Requirements Specification*




VIT
Vellore Institute of Technology

# SafeShe

*Intelligent Multi-Agent Transit Safety & Emergency Orchestration Platform*

| Field | Details |
|---|---|
| Project Title | SafeShe — Intelligent Multi-Agent Transit Safety & Emergency Orchestration Platform |
| Student Name | Keshav Kumar |
| Registration Number | 24BAI1301 |
| Course | Software Engineering |
| Faculty | Dr. A. ILAVENDHAN |
| Department | SCOPE |

*Document Reference: SAFESHE-SRS-FINAL-2026*

*Standards Compliance: IEEE Std 830-1998 (SRS) · OMG UML 2.5.1 · ProjectLibre WBS Standard*

# Table of Contents

*(In Word: right-click below and choose "Update Field", or press Ctrl+A then F9, to populate page numbers.)*

Table of Contents ..... 2
1. Introduction ..... 4
1.1 Purpose of the Document ..... 4
1.2 Scope of the System ..... 4
1.3 Definitions, Acronyms, and Abbreviations ..... 4
1.4 References ..... 4
1.5 Overview of the Document ..... 5
2. Requirement Engineering Process ..... 5
2.1 Feasibility Study ..... 5
2.2 Requirement Elicitation and Analysis ..... 5
2.3 Requirement Validation & Verification ..... 5
2.4 Requirement Management Plan ..... 5
3. Overall Description ..... 6
3.1 Product Perspective ..... 6
3.2 Product Functions ..... 6
3.3 User Classes and Characteristics ..... 6
3.4 Operating Environment ..... 6
3.5 Design and Implementation Constraints ..... 6
3.6 Assumptions and Dependencies ..... 6
4. Requirements Specification ..... 7
4.1 Functional Requirements ..... 7
4.2 Non-Functional Requirements (NFRs) ..... 7
4.3 Domain Requirements ..... 8
4.4 System Requirements ..... 8
4.4.1 User Interfaces ..... 8
4.4.2 Hardware Interfaces ..... 8
4.4.3 Software Interfaces ..... 8
4.4.4 Communication Interfaces ..... 8
5. Work Breakdown Structure (WBS) ..... 9
5.1 Level 1 — Major Phases ..... 9
5.2 Level 2 — Tasks under Each Phase ..... 9
5.3 WBS Diagram Representation ..... 10
6. System Models (UML Diagrams) ..... 13
6.1 Use Case Diagram ..... 13
6.2 Activity Diagram(s) ..... 14
6.3 Class Diagram ..... 15
6.4 Sequence Diagram(s) ..... 16
6.5 State Machine Diagram ..... 17

6.6 Component Diagram ..... 18
6.7 Deployment Diagram ..... 19
7. Data Flow Modeling ..... 20
7.1 Level 0 DFD (Context Diagram) ..... 20
7.2 Level 1 DFD ..... 20
8. Risk Analysis ..... 20
8.1 Risk Identification ..... 20
8.2 Risk Assessment Matrix (Probability vs. Impact) ..... 21
8.3 Risk Mitigation Strategies ..... 21
9. Testing Strategy ..... 22
9.1 Test Plan ..... 22
9.2 Unit Testing ..... 22
9.3 Integration Testing ..... 22
9.4 System Testing ..... 23
9.4.1 Security & Performance ..... 23
9.5 Acceptance Testing ..... 23
10. Glossary ..... 23
11. Appendices ..... 24
Appendix A — References ..... 24
Appendix B — Entity–Relationship (ER) Diagram & Data Dictionary ..... 25
Appendix C — Requirement Traceability Matrix (RTM) ..... 25
Appendix D — REST & WebSocket API Catalogue ..... 26
Appendix E — Diagram & Submission Checklist ..... 27

# 1. Introduction

## 1.1 Purpose of the Document

This document specifies the complete software requirements for the SafeShe Multimodal Transit Safety and Emergency Orchestration System. It defines the functional and non-functional behaviour of the platform in sufficient detail for developers, testers, project supervisors, and academic evaluators to build, verify, and assess the system without ambiguity. This SRS is prepared in conformance with IEEE Std 830-1998 and follows the chapter structure prescribed for this course's final submission.

## 1.2 Scope of the System

SafeShe is an intelligent personal-safety web and mobile navigation platform for commuters travelling on foot, by cab, or by public transit. The system evaluates candidate routes by combining live road-network topology, real-time weather, ambient lighting estimates, historical crime density, and crowdsourced hazard reports into a single machine-learning-derived safety score in the range [0, 100].

Beyond route planning, SafeShe provides continuous live GPS telemetry monitoring during an active journey, automatic hazard-aware rerouting, an instant one-tap emergency SOS broadcast with SMS and live-tracking fallback, a crowdsourced hazard-reporting layer, a conversational AI safety assistant, and a city-wide ambient safety dashboard.

The scope covers the web client, the FastAPI backend, the multi-agent intelligence layer, the ML safety-scoring pipeline, and the MongoDB persistence layer. It excludes law-enforcement control-room systems and any future wearable companion application (see Future Scope, Section 3.6).

## 1.3 Definitions, Acronyms, and Abbreviations

Frequently used terms are defined below; the complete glossary of domain and technical terms is provided in Chapter 10.

| Term | Definition |
|---|---|
| SRS | Software Requirements Specification |
| JIC | Journey Intelligence Coordinator — internal orchestrator for routing, scoring and explanation |
| DTO | Data Transfer Object |
| JWT | JSON Web Token |
| GeoJSON | Geospatial data interchange format representing points, lines and polygons as JSON |
| 2dsphere Index | MongoDB index type enabling efficient geospatial queries |
| IDOR | Insecure Direct Object Reference — a security flaw allowing access to another user's resource |

## 1.4 References

- IEEE Std 830-1998 — IEEE Recommended Practice for Software Requirements Specifications.
- OMG Unified Modeling Language (OMG UML) Superstructure, Version 2.5.1.
- RFC 7519 — JSON Web Token (JWT) Architecture and Validation; RFC 6455 — The WebSocket Protocol.
- W3C Web Content Accessibility Guidelines (WCAG) 2.1.
- Full reference list is consolidated in Appendix A.

### 1.5 Overview of the Document

Chapter 2 describes the requirement engineering process. Chapter 3 gives the overall product description. Chapter 4 specifies the functional, non-functional, domain, and system requirements. Chapter 5 presents the Work Breakdown Structure. Chapter 6 provides the UML system-model specification for construction in StarUML. Chapter 7 models the system's data flow. Chapter 8 analyses project risk. Chapter 9 defines the testing strategy. Chapter 10 is a glossary, and Chapter 11 contains the appendices — the ER model, the Requirement Traceability Matrix, and the API catalogue.

## 2. Requirement Engineering Process

### 2.1 Feasibility Study

- **Technical:** All core technologies — FastAPI/Python, Next.js/React, MongoDB, XGBoost — are mature, open-source, and well documented; OSRM and Nominatim can be self-hosted at no licensing cost. The existing project repository already implements the core routing, emergency, and community modules, confirming technical achievability within one academic semester.
- **Operational:** Target user groups — commuters, emergency contacts, administrators — require no specialised training; the interface uses familiar map interactions and a single-tap SOS control. Emergency contacts need only a browser, with no app install required.
- **Economic:** Development relies primarily on open-source infrastructure; the only recurring cost is the Twilio SMS gateway, which can be substituted with email/WhatsApp for a budget-constrained prototype.
- **Schedule:** The 120-working-day WBS (Chapter 5) parallelises Frontend, Backend, and ML streams converging at a single Integration milestone, making the schedule achievable within a standard academic-project timeline.

### 2.2 Requirement Elicitation and Analysis

Requirements were elicited through: (a) code-based requirement mining of the existing FastAPI routers, Pydantic models and Next.js pages to reverse-derive enforced business rules; (b) structured walkthroughs of each client page (Dashboard, Journey Planner, Live Tracking, Community, Emergency, Assistant, Profile, Settings); (c) scenario analysis of representative use (night-time walking, hazard reporting, SOS activation) to surface edge cases; (d) stakeholder role analysis across Commuter, Emergency Contact, Administrator, and the internal Journey Intelligence Coordinator; and (e) comparative review of comparable personal-safety and navigation applications to confirm completeness.

### 2.3 Requirement Validation & Verification

Each functional requirement was validated against four artefacts before acceptance into this baseline: a REST/WebSocket endpoint with request/response DTO, a database collection or embedded field, at least one UML diagram (Chapter 6), and at least one test case (Chapter 9). This four-way cross-check is recorded as the Requirement Traceability Matrix (Appendix C). Every NFR was written with a measurable, testable threshold (e.g., "under 1500 ms", "99.9% uptime") so verification does not rely on subjective judgement, and requirements were checked for mutual consistency.

### 2.4 Requirement Management Plan

This document (tag SAFESHE-SRS-FINAL-2026) forms the requirements baseline. Identifiers (FR-xxx, NFR-xxx) are fixed for the project's life so downstream UML, API, database, and test artefacts never need renumbering. Any change request is logged, re-validated using the Section 2.3 four-way check, and reflected in the RTM, affected UML diagram(s), and test case(s) before acceptance. The SRS is version-controlled alongside the source code.

# 3. Overall Description

## 3.1 Product Perspective

SafeShe is a decoupled, multi-tier system: an event-driven single-page client, an asynchronous FastAPI microservice backend, an autonomous multi-agent intelligence layer, an offline spatial data-science pipeline, and a distributed MongoDB repository. The client (Next.js / React / MapLibre GL) never talks to external providers or the database directly — every request is brokered through the backend over HTTPS/WebSockets. The backend routes requests through a service layer to repositories and delegates safety intelligence to the Journey Intelligence Coordinator, which orchestrates specialised cognitive agents (Routing, Weather, Community, Emergency, Transit), an ML scoring pipeline, and external providers (OSRM, Nominatim, OpenWeatherMap, Twilio).

## 3.2 Product Functions

- Multimodal safe route planning incorporating spatial hazards, weather, and lighting.
- Autonomous agent orchestration for context aggregation and route explanation.
- Live GPS telemetry tracking with automated route-deviation and hazard-proximity detection.
- Instant emergency SOS broadcasting with live coordinate streaming and SMS fallback.
- Crowdsourced geospatial safety incident reporting with upvote/downvote verification.
- Context-aware conversational AI assistant for localized transit safety queries.
- City-wide ambient safety scoring and regional risk heatmaps.

## 3.3 User Classes and Characteristics

- **Standard Commuter (Primary):** seeks optimized, safe transit paths and quick access to emergency utilities; requires a simple, high-visibility UI.
- **Emergency Contact (Observer):** monitors real-time location feeds when alerted; requires zero-install web tracker access.
- **Community Contributor:** submits localized hazard reports and validates peer reports.
- **System Administrator / Analyst:** audits model telemetry and moderates community reports.

## 3.4 Operating Environment

- Server: Linux (Ubuntu 22.04 LTS) or Windows Server, Python 3.10+, Uvicorn 0.29+, Node.js 20+, MongoDB Community Edition 7.0+.
- Client: Chrome 120+, Firefox 120+, Safari 17+, Edge 120+, with WebGL 2.0 and the Geolocation API enabled.

## 3.5 Design and Implementation Constraints

- The backend must operate asynchronously without blocking the event loop.
- The system must degrade gracefully when external APIs time out, rather than surfacing an unhandled server error.
- The client is restricted to browser-native geolocation and WebGL rendering; no native mobile SDK is in scope.
- All secrets (JWT_SECRET, MONGO_URI, WEATHER_API_KEY) must be injected via environment variables.

## 3.6 Assumptions and Dependencies

- Client devices possess active network connectivity and GPS hardware accurate to 15 metres or better.

- The system depends on the continued availability of OSRM, Nominatim, OpenWeatherMap, and Twilio (see Chapter 8 for mitigation if any becomes unavailable).
- **Future Scope (not in current baseline):** wearable companion apps, computer-vision-based street-lighting analysis, and secure webhook integration with police control rooms.

# 4. Requirements Specification

Each requirement is numbered for direct traceability to the API catalogue, database schema, UML diagrams, and test cases (Appendix C, Requirement Traceability Matrix).

## 4.1 Functional Requirements

| ID | Requirement | Actor(s) | Key Business Rule |
|---|---|---|---|
| FR-001 | User Registration | End User | Unique email; BCrypt password hash |
| FR-002 | User Authentication | End User | HTTP 401 on invalid credentials; JWT expires in 24h |
| FR-003 | Preference Management | End User | User may only mutate own preferences |
| FR-004 | Emergency Contact Management | End User | Maximum 5 contacts per account |
| FR-005 | Safe Route Planning | End User, JIC | 1–3 routes; score in [0,100]; severe crime near segment caps score at 30 |
| FR-006 | Journey Lifecycle Transition | End User, Journey Svc | Transitions strictly follow the state machine (§6.5) |
| FR-007 | Live Telemetry Monitoring | Device, Telemetry Engine | Polled every 5s or pushed via WebSocket; distance via Haversine |
| FR-008 | Dynamic Route Rerouting | JIC | Reroute alert generated; user confirmation required |
| FR-009 | Emergency SOS Dispatch | End User, Emergency Svc | Session created; SMS dispatch triggered within 300 ms |
| FR-010 | Real-Time SOS Live Stream | Device, Observer, WS Server | Authenticated via session_id; terminates on resolution |
| FR-011 | Crowdsourced Hazard Reporting | End User, Community Svc | Duplicate reports within 50 m by same user within 1h rejected |
| FR-012 | Geospatial Hazard Retrieval | End User, Community Svc | MongoDB $near on 2dsphere index; default radius 5000 m |
| FR-013 | Conversational AI Assistant | End User, Assistant Agent | Queries nearby safe zones; concise formatted responses |
| FR-014 | City Safety Dashboard | End User, Dashboard Agent | Aggregates weather, open reports, and active journeys |
| FR-015 | Historical Activity Audit | End User, Journey Svc | Users may only query their own historical journeys |

## 4.2 Non-Functional Requirements (NFRs)

| ID | Category | Requirement Statement |
|---|---|---|
| NFR-001 | Performance | Route generation and scoring pipeline completes within 1500 ms under standard load |
| NFR-002 | Performance | SOS dispatch API responds and initiates async notification within 300 ms |

| ID | Category | Requirement Statement |
|---|---|---|
| NFR-008 | Performance | Geospatial queries on up to 1,000,000 records execute under 20 ms (2dsphere index) |
| NFR-003 | Security | TLS 1.3 in transit; BCrypt (work factor 12) for passwords; JWT signed with HMAC-SHA256 |
| NFR-004 | Security | Owner-ID validation on every endpoint prevents IDOR; CORS restricted; rate limiting 100/min (10/min on auth) |
| NFR-005 | Reliability | Circuit-breaking / graceful fallback when external providers time out (>5000 ms) |
| NFR-006 | Availability | Core API and WebSocket engine maintain 99.9% uptime per month |
| NFR-007 | Scalability | WebSocket architecture supports up to 10,000 concurrent connections |
| NFR-010 | Maintainability | Clean Architecture — business logic, repositories, coordinators, providers fully decoupled |
| NFR-011 | Portability | Runs unmodified on Linux/Windows with Python 3.10+, MongoDB 7.0+; client on evergreen browsers |
| NFR-009 | Usability | WCAG 2.1 AA compliance; 64×64px minimum SOS touch target |

## 4.3 Domain Requirements

Requirements arising from the geospatial safety-navigation domain itself, constraining how the functional requirements above are implemented.

| ID | Requirement |
|---|---|
| DR-001 | GeoJSON coordinates ordered [longitude, latitude] (RFC 7946); radius queries use the 2dsphere index. |
| DR-002 | Aggregate safety score normalised to [0,100]; a route intersecting a HIGH/CRITICAL hazard report is capped at 30. |
| DR-003 | An active SOS live-tracking channel remains reachable even if the SMS provider is unavailable (fail-open). |
| DR-004 | GPS streams for completed journeys are generalised after 30 days; emergency logs retained 90 days for audit. |
| DR-005 | Emergency contact phone numbers stored/dispatched in E.164 format. |
| DR-006 | A hazard report is surfaced to scoring only if net votes ≥ 0 or it is younger than 24 hours. |

## 4.4 System Requirements

### *4.4.1 User Interfaces*

- Single Page Application in Next.js 16 / React 19; map rendering via MapLibre GL JS (dark/light styles).
- Real-time toast alerts for hazard, reroute, and SOS events.
- Responsive layouts for mobile (360–480px), tablet (768–1024px), and desktop (1280px+).

### *4.4.2 Hardware Interfaces*

- Client GPS sensor streams coordinates via the W3C Geolocation API.
- Accelerometer/gyroscope captures motion anomalies to trigger gesture-based SOS.

### *4.4.3 Software Interfaces*

- Nominatim (geocoding), OSRM (routing), OpenWeatherMap (weather), Twilio (SMS) — all via HTTPS REST.
- MongoDB Community Server 7.0+ over the native wire protocol.

### *4.4.4 Communication Interfaces*

- HTTPS (Port 443) — REST API with TLS 1.3.

- WSS (Port 443) — secure WebSocket for real-time telemetry and emergency broadcast.

# 5. Work Breakdown Structure (WBS)

Provided as structured source data for direct recreation in ProjectLibre. Durations assume a single academic team working a 120-working-day (approx. one-semester) timeline.

## 5.1 Level 1 — Major Phases

| WBS No. | Phase | Duration | Milestone |
|---|---|---|---|
| 1.1 | Requirements Engineering | 12d | SRS Approved |
| 1.2 | System Analysis & Modeling | 14d | Analysis Done |
| 1.3 | Architectural & Database Design | 12d | Design Freeze |
| 1.4 | Frontend Development (Next.js/React) | 28d | Frontend MVP |
| 1.5 | Backend Development (FastAPI/Motor) | 30d | Backend MVP |
| 1.6 | Machine Learning Pipeline Development | 25d | Model Ready |
| 1.7 | Database Development & Index Provisioning | 8d | DB Provisioned |
| 1.8 | Subsystem Integration | 15d | Integrated |
| 1.9 | Verification & Quality Assurance | 18d | QA Signoff |
| 1.10 | Deployment & DevOps | 10d | Deployed |
| 1.11 | Project Documentation | 12d | Docs Ready |
| 1.12 | Final Academic Submission | 3d | Submission Final |

## 5.2 Level 2 — Tasks under Each Phase

Task descriptions, durations and predecessors for each Level-1 phase (WBS numbering carried forward from §5.1).

| WBS No. | Task | Duration | Predecessor(s) |
|---|---|---|---|
| 1.1.1 | Stakeholder Elicitation & Domain Discovery | 4d | — |
| 1.1.2 | Functional & Non-Functional Requirements (SRS) | 5d | 1.1.1 |
| 1.1.3 | API Contract Specification | 3d | 1.1.2 |
| 1.2.1 | UML Structural Modeling (Class, Component, Package) | 5d | 1.1.3 |
| 1.2.2 | UML Behavioral Modeling (Sequence, State, Activity) | 5d | 1.2.1 |
| 1.2.3 | Requirement Traceability Matrix (RTM) | 4d | 1.2.2 |
| 1.3.1 | Database ER Modeling & Index Strategy | 4d | 1.2.3 |
| 1.3.2 | Multi-Agent Cognitive Architecture Design | 4d | 1.3.1 |
| 1.3.3 | UI/UX Design System & Theme Tokens | 4d | 1.3.1 |
| 1.4.1 | Map Engine Integration & Route Rendering | 7d | 1.3.3 |
| 1.4.2 | Journey Planning & Multi-Route Selection UI | 7d | 1.4.1 |
| 1.4.3 | Live Telemetry Dashboard & Active Navigation | 7d | 1.4.2 |
| 1.4.4 | Emergency SOS Modal & WebSocket Client | 7d | 1.4.3 |
| 1.5.1 | MongoDB Connection & Repository CRUD Layer | 5d | 1.3.1 |
| 1.5.2 | Provider Layer (OSRM, OpenWeather, Nominatim) | 7d | 1.5.1 |

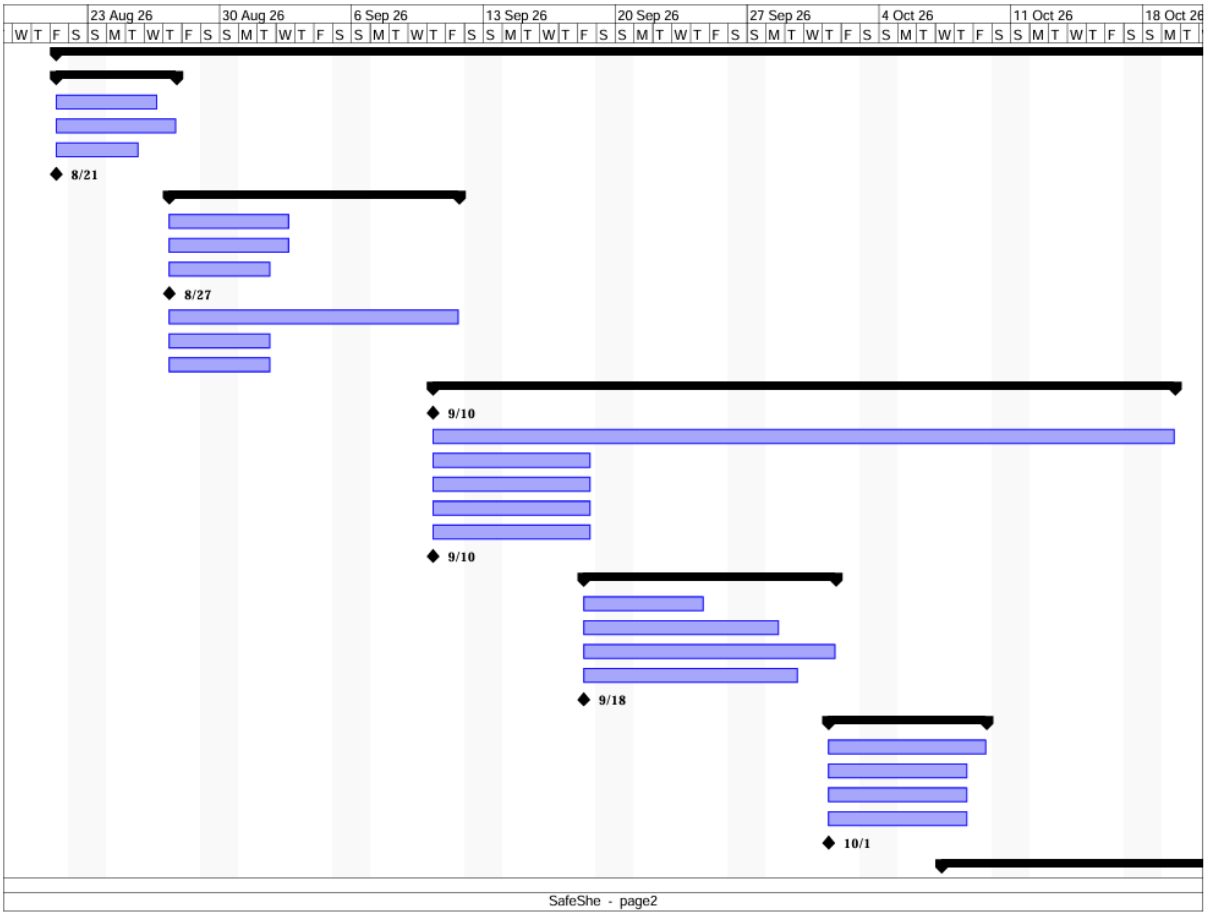
| WBS No. | Task | Duration | Predecessor(s) |
|---|---|---|---|
| 1.5.3 | Journey Intelligence Coordinator & Agents | 10d | 1.5.2 |
| 1.5.4 | REST Routers & WebSocket Gateway Implementation | 8d | 1.5.3 |
| 1.6.1 | Road Segmentation & POI Spatial Joins | 7d | 1.3.1 |
| 1.6.2 | Population & Crime Data Harmonization | 6d | 1.6.1 |
| 1.6.3 | Feature Engineering (Lighting, CCTV proxies) | 6d | 1.6.2 |
| 1.6.4 | XGBoost Model Training & Validation | 6d | 1.6.3 |
| 1.7.1 | Collection Creation & 2dsphere Indexes | 4d | 1.3.1 |
| 1.7.2 | Seed Scripts & Migration Harness | 4d | 1.7.1 |
| 1.8.1 | Mount Trained ML Model into Backend | 4d | 1.6.4, 1.5.4 |
| 1.8.2 | Wire Frontend Services to Backend Routers | 6d | 1.4.4, 1.8.1 |
| 1.8.3 | Live WebSocket Channel Verification | 5d | 1.8.2 |
| 1.9.1 | Backend Unit & Integration Testing | 6d | 1.8.3 |
| 1.9.2 | System & Security Testing | 6d | 1.9.1 |
| 1.9.3 | User Acceptance Testing (UAT) | 6d | 1.9.2 |
| 1.10.1 | Containerization (Frontend, Backend, DB) | 5d | 1.9.3 |
| 1.10.2 | CI/CD Pipeline Setup & Health Monitoring | 5d | 1.10.1 |
| 1.11.1 | Academic SRS & UML Diagram Finalization | 6d | 1.9.3 |
| 1.11.2 | API Catalogue & Final Report Assembly | 6d | 1.11.1 |
| 1.12 | Final Academic Submission | 3d | 1.10, 1.11 |

## 5.3 WBS Diagram Representation

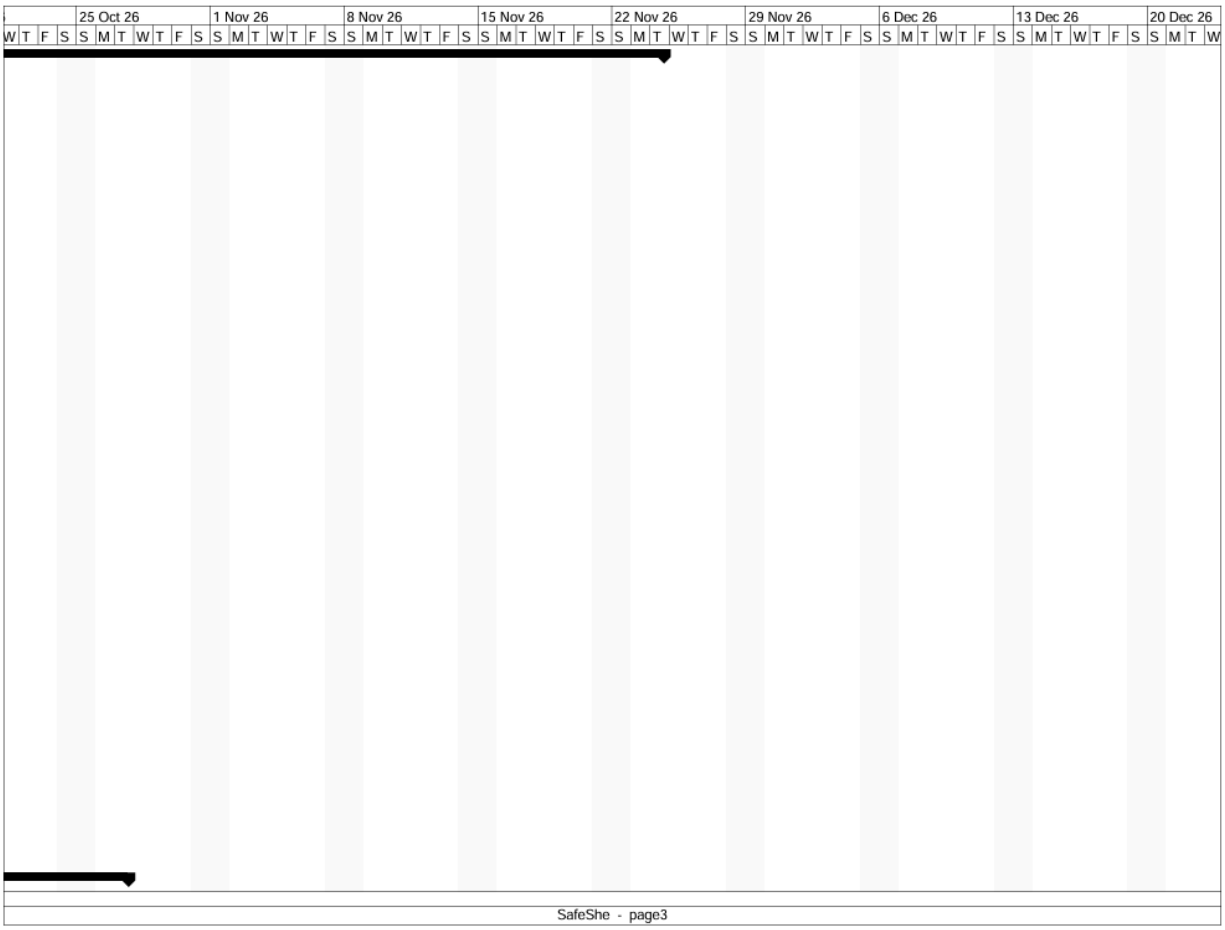
This hierarchy is to be recreated as a Gantt chart and network diagram directly in ProjectLibre using the WBS numbers, durations, and predecessor links given. Indicative critical path: Requirements Engineering (1.1) → System Analysis & Modeling (1.2) → Architectural & Database Design (1.3) → Backend Development (1.5, 30d) → Subsystem Integration (1.8) → Verification & QA (1.9) → Deployment & DevOps (1.10) → Project Documentation (1.11) → Final Academic Submission (1.12). Frontend (1.4, 28d) and ML (1.6, 25d) run in parallel with Backend and do not constrain the critical path.

| | | Name | Duration | Start | Finish | Predecessors | Resource Names |
|---|---|---|---|---|---|---|---|
| 1 | | **SafeShe Project** | **68 days** | **8/21/26, 8:00AM** | **11/24/26, 5:00PM** | | |
| 2 | | **Requirements Engine...** | **5 days** | **8/21/26, 8:00AM** | **8/27/26, 5:00PM** | | |
| 3 | | Stakeholder Elicitation ... | 4 days | 8/21/26, 8:00AM | 8/26/26, 5:00PM | | |
| 4 | | Functional & Non-Func... | 5 days | 8/21/26, 8:00AM | 8/27/26, 5:00PM | | |
| 5 | | API Contract Specifica... | 3 days | 8/21/26, 8:00AM | 8/25/26, 5:00PM | | |
| 6 | | SRS Approved | 0 days | 8/21/26, 8:00AM | 8/21/26, 8:00AM | | |
| 7 | | **System Analysis & M...** | **12 days** | **8/27/26, 8:00AM** | **9/11/26, 5:00PM** | | |
| 8 | | UML Structural Modeling | 5 days | 8/27/26, 8:00AM | 9/2/26, 5:00PM | | |
| 9 | | UML Behavioral Modeling | 5 days | 8/27/26, 8:00AM | 9/2/26, 5:00PM | | |
| 10 | | Requirement Traceabili... | 4 days | 8/27/26, 8:00AM | 9/1/26, 5:00PM | | |
| 11 | | Analysis Done | 0 days | 8/27/26, 8:00AM | 8/27/26, 8:00AM | | |
| 12 | | Architectural & Databa... | 12 days | 8/27/26, 8:00AM | 9/11/26, 5:00PM | | |
| 13 | | Database ER Modeling ... | 4 days | 8/27/26, 8:00AM | 9/1/26, 5:00PM | | |
| 14 | | Multi-Agent Cognitive ... | 4 days | 8/27/26, 8:00AM | 9/1/26, 5:00PM | | |
| 15 | | **UI/UX Design System...** | **28 days** | **9/10/26, 8:00AM** | **10/19/26, 5:00PM** | | |
| 16 | | Design Freeze | 0 days | 9/10/26, 8:00AM | 9/10/26, 8:00AM | | |
| 17 | | Frontend Development | 28 days | 9/10/26, 8:00AM | 10/19/26, 5:00PM | | |
| 18 | | Map Engine Integratio... | 7 days | 9/10/26, 8:00AM | 9/18/26, 5:00PM | | |
| 19 | | Journey Planning & Mu... | 7 days | 9/10/26, 8:00AM | 9/18/26, 5:00PM | | |
| 20 | | Live Telemetry Dashbo... | 7 days | 9/10/26, 8:00AM | 9/18/26, 5:00PM | | |
| 21 | | Emergency SOS Modal ... | 7 days | 9/10/26, 8:00AM | 9/18/26, 5:00PM | | |
| 22 | | Frontend MVP | 0 days | 9/10/26, 8:00AM | 9/10/26, 8:00AM | | |
| 23 | | **Backend Development** | **10 days** | **9/18/26, 8:00AM** | **10/1/26, 5:00PM** | | |
| 24 | | MongoDB Connection ... | 5 days | 9/18/26, 8:00AM | 9/24/26, 5:00PM | | |
| 25 | | Provider Layer (OSRM,... | 7 days | 9/18/26, 8:00AM | 9/28/26, 5:00PM | | |
| 26 | | Journey Intelligence C... | 10 days | 9/18/26, 8:00AM | 10/1/26, 5:00PM | | |
| 27 | | REST Routers & WebS... | 8 days | 9/18/26, 8:00AM | 9/29/26, 5:00PM | | |
| 28 | | Backend MVP | 0 days | 9/18/26, 8:00AM | 9/18/26, 8:00AM | | |
| 29 | | **Machine Learning Pip...** | **7 days** | **10/1/26, 8:00AM** | **10/9/26, 5:00PM** | | |
| 30 | | Road Segmentation & ... | 7 days | 10/1/26, 8:00AM | 10/9/26, 5:00PM | | |
| 31 | | Population & Crime Dat... | 6 days | 10/1/26, 8:00AM | 10/8/26, 5:00PM | | |
| 32 | | Feature Engineering | 6 days | 10/1/26, 8:00AM | 10/8/26, 5:00PM | | |
| 33 | | XGBoost Model Trainin... | 6 days | 10/1/26, 8:00AM | 10/8/26, 5:00PM | | |
| 34 | | Model Ready | 0 days | 10/1/26, 8:00AM | 10/1/26, 8:00AM | | |
| 35 | | **Database Developme...** | **15 days** | **10/7/26, 8:00AM** | **10/27/26, 5:00PM** | | |

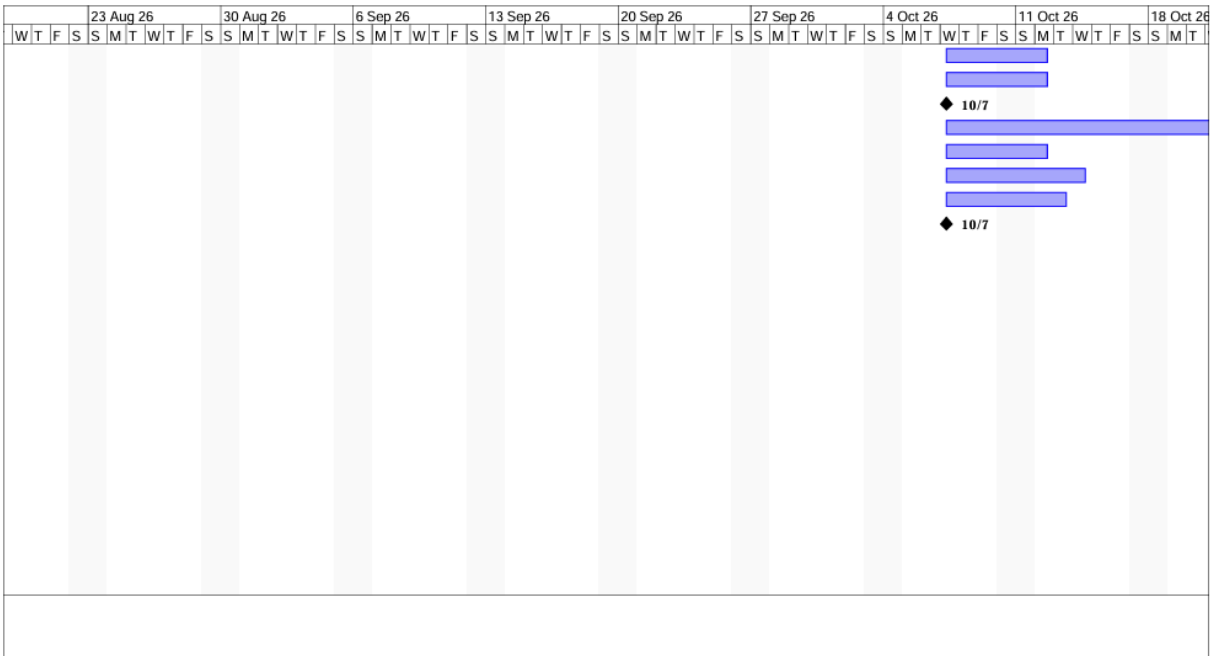
SafeShe - page1

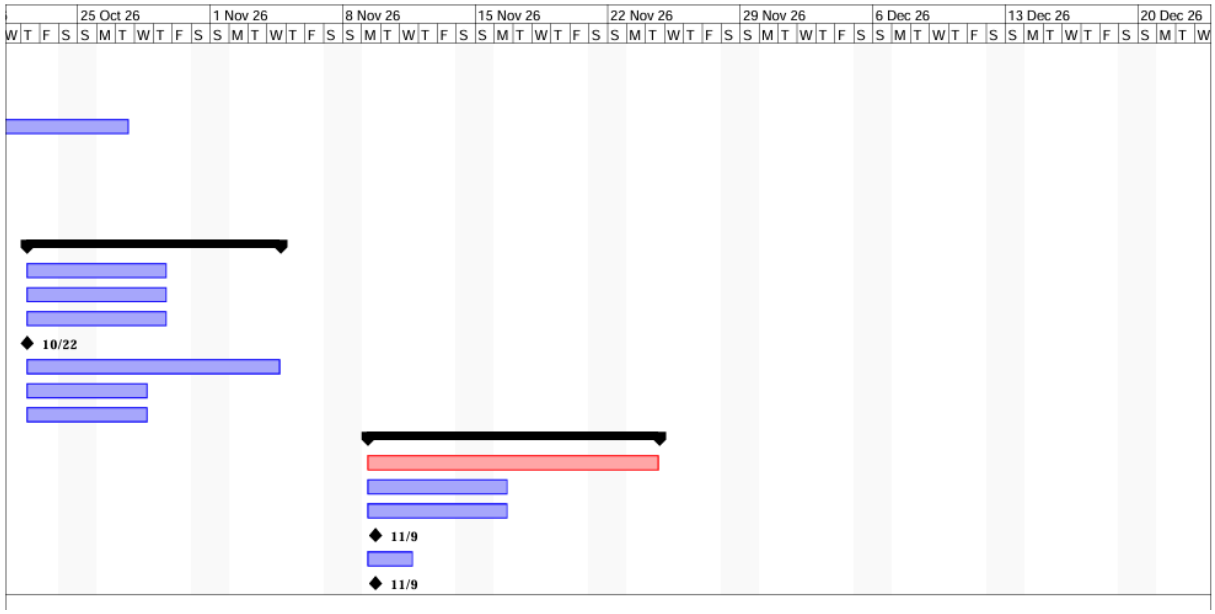



SafeShe - page2




| | | Name | Duration | Start | Finish | Predecessors | Resource Names |
|---|---|---|---|---|---|---|---|
| 36 | | Collection Creation & 2... | 4 days | 10/7/26, 8:00AM | 10/12/26, 5:00PM | | |
| 37 | | Seed Scripts & Migratio... | 4 days | 10/7/26, 8:00AM | 10/12/26, 5:00PM | | |
| 38 | | DB Provisioned | 0 days | 10/7/26, 8:00AM | 10/7/26, 8:00AM | | |
| 39 | | Subsystem Integration | 15 days | 10/7/26, 8:00AM | 10/27/26, 5:00PM | | |
| 40 | | Mount Trained ML Mod... | 4 days | 10/7/26, 8:00AM | 10/12/26, 5:00PM | | |
| 41 | | Wire Frontend Service... | 6 days | 10/7/26, 8:00AM | 10/14/26, 5:00PM | | |
| 42 | | Live WebSocket Chann... | 5 days | 10/7/26, 8:00AM | 10/13/26, 5:00PM | | |
| 43 | | Integrated | 0 days | 10/7/26, 8:00AM | 10/7/26, 8:00AM | | |
| 44 | | **Verification & Quality ...** | **10 days** | **10/22/26, 8:00AM** | **11/4/26, 5:00PM** | | |
| 45 | | Backend Unit & Integr... | 6 days | 10/22/26, 8:00AM | 10/29/26, 5:00PM | | |
| 46 | | System & Security Tes... | 6 days | 10/22/26, 8:00AM | 10/29/26, 5:00PM | | |
| 47 | | User Acceptance Testing | 6 days | 10/22/26, 8:00AM | 10/29/26, 5:00PM | | |
| 48 | | QA Signoff | 0 days | 10/22/26, 8:00AM | 10/22/26, 8:00AM | | |
| 49 | | Deployment & DevOps | 10 days | 10/22/26, 8:00AM | 11/4/26, 5:00PM | | |
| 50 | | Containerization | 5 days | 10/22/26, 8:00AM | 10/28/26, 5:00PM | | |
| 51 | | CI/CD Pipeline Setup &... | 5 days | 10/22/26, 8:00AM | 10/28/26, 5:00PM | | |
| 52 | | **Deployed** | **12 days** | **11/9/26, 8:00AM** | **11/24/26, 5:00PM** | | |
| 53 | | Project Documentation | 12 days | 11/9/26, 8:00AM | 11/24/26, 5:00PM | | |
| 54 | | Academic SRS & UML D.. | 6 days | 11/9/26, 8:00AM | 11/16/26, 5:00PM | | |
| 55 | | API Catalogue & Final ... | 6 days | 11/9/26, 8:00AM | 11/16/26, 5:00PM | | |
| 56 | | Docs Ready | 0 days | 11/9/26, 5:00PM | 11/9/26, 5:00PM | | |
| 57 | | Final Academic Submis... | 3 days | 11/9/26, 8:00AM | 11/11/26, 5:00PM | | |
| 58 | | Submission Final | 0 days | 11/9/26, 5:00PM | 11/9/26, 5:00PM | | |




# 6. System Models (UML Diagrams)

This chapter provides structured source data for each of the seven required UML diagrams, to be constructed manually in StarUML. A placeholder marks where each finished StarUML export is to be inserted.

## 6.1 Use Case Diagram

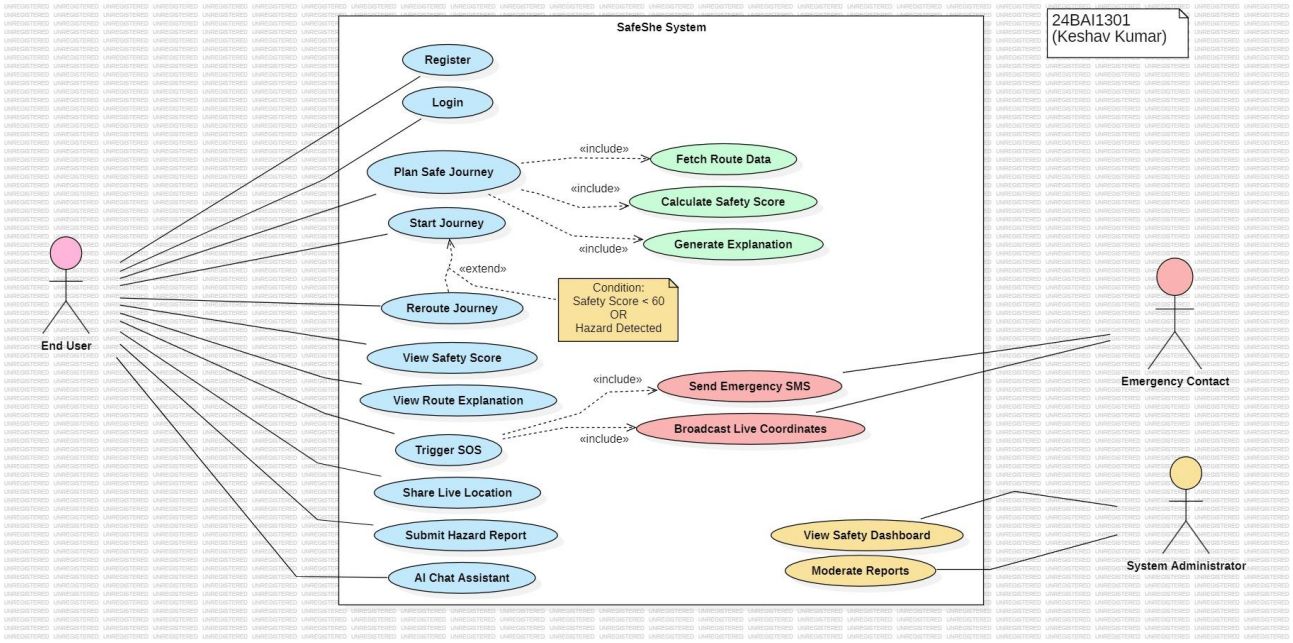
**Purpose:** Models the functional interactions between actors and the SafeShe platform across navigation, emergency response, community reporting, and administrative operations.

**Key Elements — Actors:**

- End User (Commuter) — Primary; plans journeys, navigates, triggers SOS, reports hazards, uses the AI assistant.
- Emergency Contact — Secondary; receives distress SMS and monitors live tracking.

- System Administrator — Monitors system activities through the dashboard and moderates community reports.

**Brief Description:** The Use Case Diagram consists of eighteen use cases covering authentication, journey planning, safety monitoring, emergency response, community reporting, AI-assisted interaction, and administrative management. The “Plan Safe Journey” use case includes route retrieval, safety score calculation, and route explanation generation. The “Trigger SOS” use case includes emergency SMS transmission and live coordinate broadcasting. The “Reroute Journey” use case extends “Start Journey” whenever the safety score falls below the acceptable threshold or a hazard is detected on the active route.



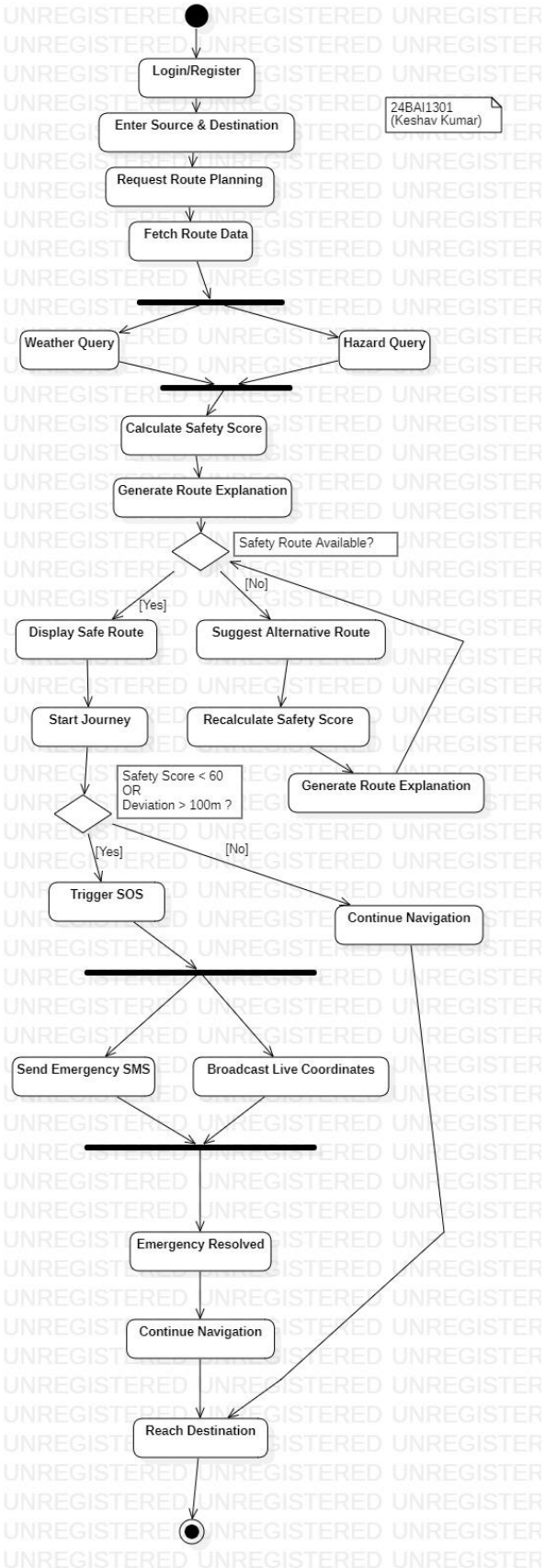

## 6.2 Activity Diagram(s)

**Purpose:** Models end-to-end procedural workflows with decision points and concurrent (fork/join) provider calls across four swimlanes.

**Key Elements — Swimlanes & Workflows:**

- Swimlanes: Client SPA, Backend Service, Intelligence & Agents, External Providers.
- Eight workflows: Registration, Login, Safe Route Planning, Start Journey, Live Monitoring & Rerouting, Emergency SOS Trigger, Hazard Incident Reporting, Conversational AI Chat.

**Brief Description:** Safe Route Planning forks into parallel weather and hazard-pin queries that join before feature engineering and ML scoring. Emergency SOS forks into parallel SMS dispatch and WebSocket streaming that join before broadcasting to observers. Key decisions: "Email exists?" (Registration), "Password valid?" (Login), "Safety Score < 60 or Deviation > 100 m?" (Live Monitoring, triggers reroute).


Login/Register
Enter Source & Destination
24BAI1301
(Keshav Kumar)
Request Route Planning
Fetch Route Data
Weather Query
Hazard Query
Calculate Safety Score
Generate Route Explanation
Safety Route Available?
[Yes]
[No]
Display Safe Route
Suggest Alternative Route
Start Journey
Recalculate Safety Score
Generate Route Explanation
Safety Score < 60
OR
Deviation > 100m ?
[Yes]
[No]
Trigger SOS
Continue Navigation
Send Emergency SMS
Broadcast Live Coordinates
Emergency Resolved
Continue Navigation
Reach Destination

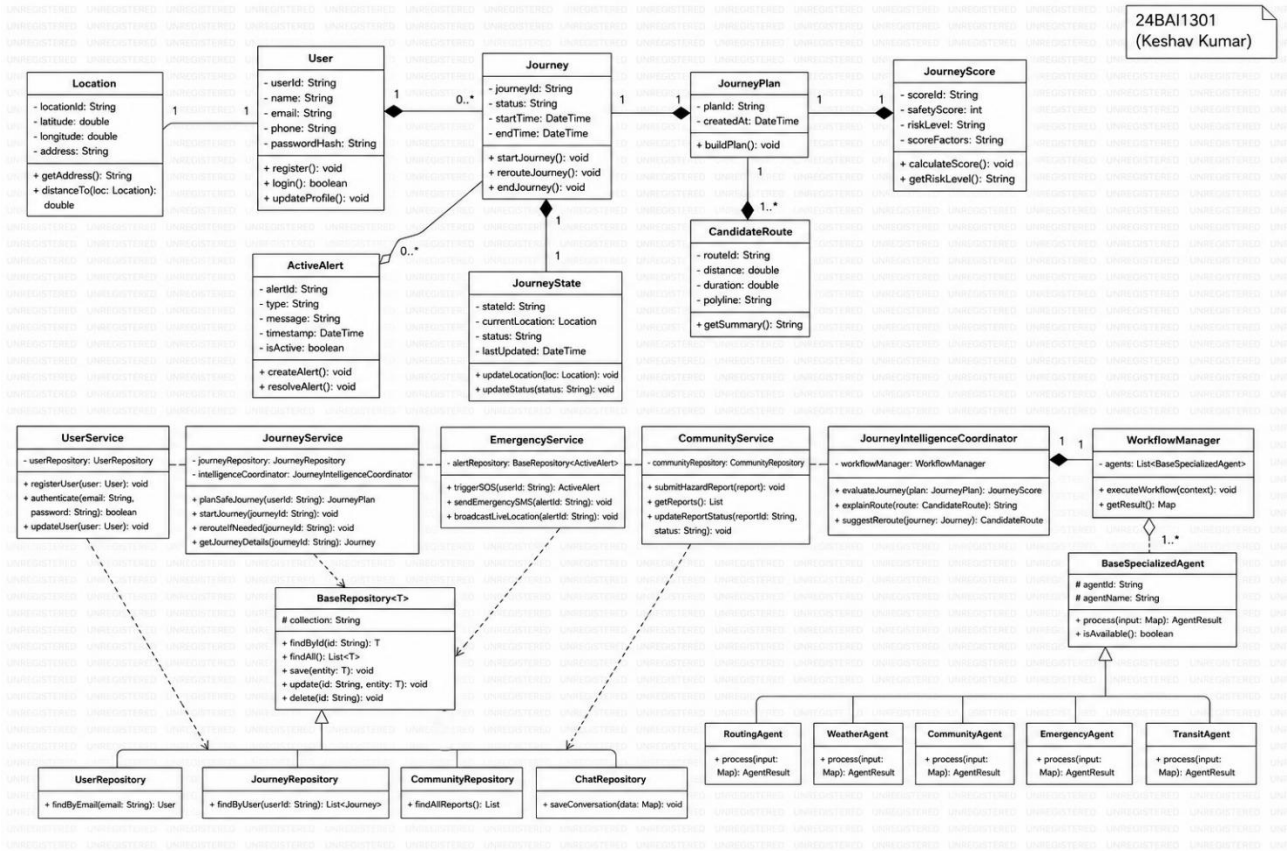
## 6.3 Class Diagram

**Purpose:** Represents the static object-oriented architecture — models, services, intelligence coordinators, agents, and repositories.

**Key Elements — Packages:**

- Models: Location, User, Journey, JourneyPlan, JourneyState, JourneyScore, CandidateRoute, ActiveAlert.
- Services: JourneyService, UserService, EmergencyService, CommunityService.
- Intelligence & Agents: JourneyIntelligenceCoordinator, WorkflowManager, BaseSpecializedAgent (→ RoutingAgent, WeatherAgent, CommunityAgent, EmergencyAgent, TransitAgent).
- Repositories: BaseRepository<T> (→ JourneyRepository, UserRepository, CommunityRepository, ChatRepository).

**Brief Description:** User composes 0..* Journey; Journey composes JourneyPlan and JourneyState, and aggregates ActiveAlert. JourneyPlan composes JourneyScore and one or more CandidateRoute objects. JourneyService depends on JourneyRepository and associates with JourneyIntelligenceCoordinator, which composes a WorkflowManager that aggregates one or more BaseSpecializedAgent subclasses. The repository layer follows inheritance from BaseRepository<T> through JourneyRepository, UserRepository, CommunityRepository, and ChatRepository, while specialized agents inherit from BaseSpecializedAgent to support routing, weather analysis, community intelligence, emergency handling, and transit assistance.




## 6.4 Sequence Diagram(s)

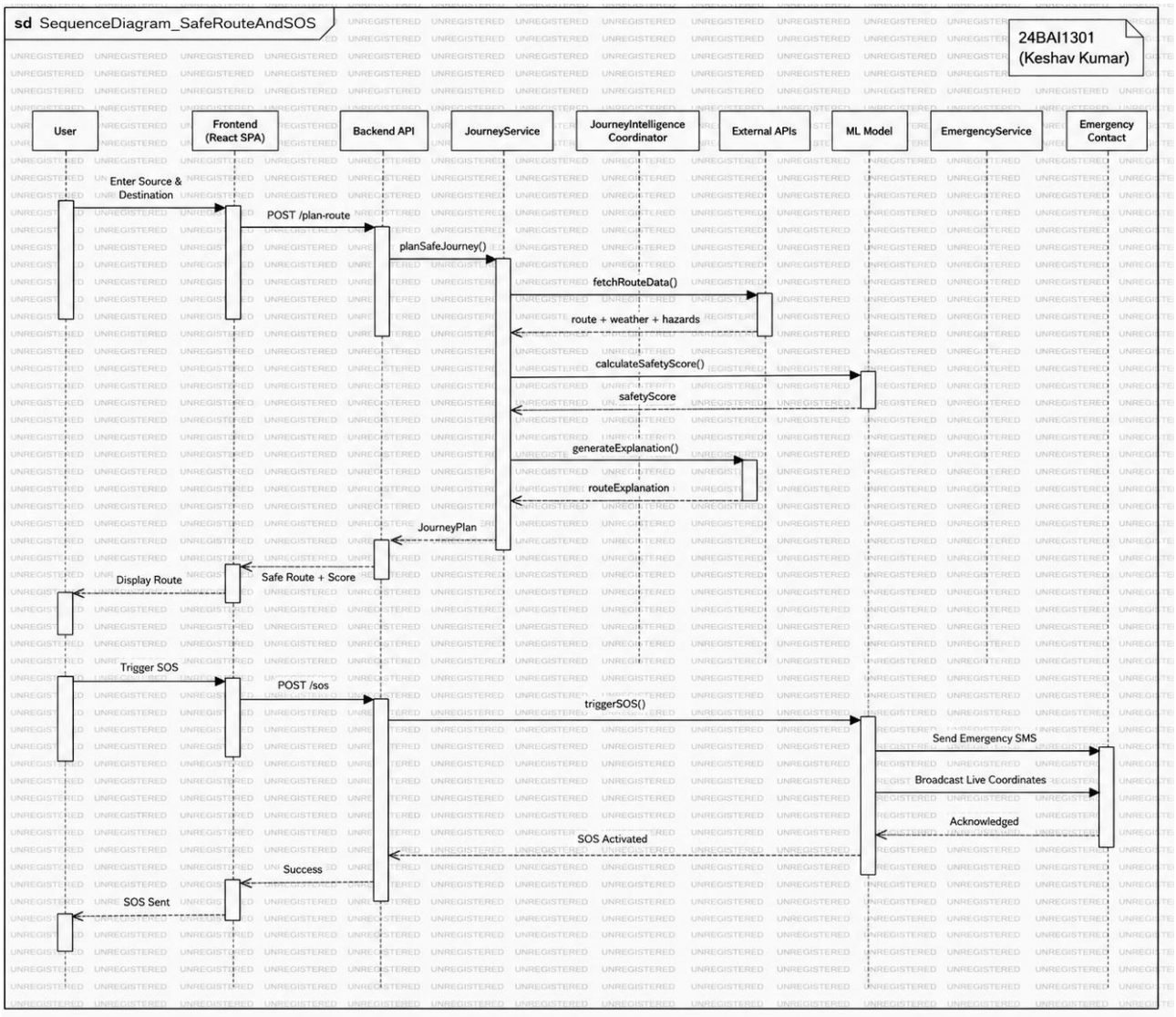
**Purpose:** Details chronological message passing, activations, and return payloads for five core workflows.

**Key Elements — Workflows & Lifelines:**

- Workflows: User Login, Safe Route Planning, Journey Start & Telemetry, Emergency SOS Activation, Hazard Report Creation.

- Representative lifelines (Route Planning): ClientApp, JourneyRouter, JourneyService, Coordinator, OSRMProvider, MLModel, MongoDB.
- Representative lifelines (Emergency SOS): ClientApp, EmergencyRouter, EmergencyService, NotificationProvider, EmergencyWSManager, ObserverClient.

**Brief Description:** Safe Route Planning: ClientApp posts to JourneyRouter → JourneyService persists a CREATED journey → Coordinator queries OSRM, engineers features, invokes the ML model, and returns a scored JourneyPlanResponse rendered on the map. Emergency SOS: ClientApp posts to EmergencyRouter → EmergencyService creates a session and dispatches SMS while the emergency WebSocket streams live GPS to observers.



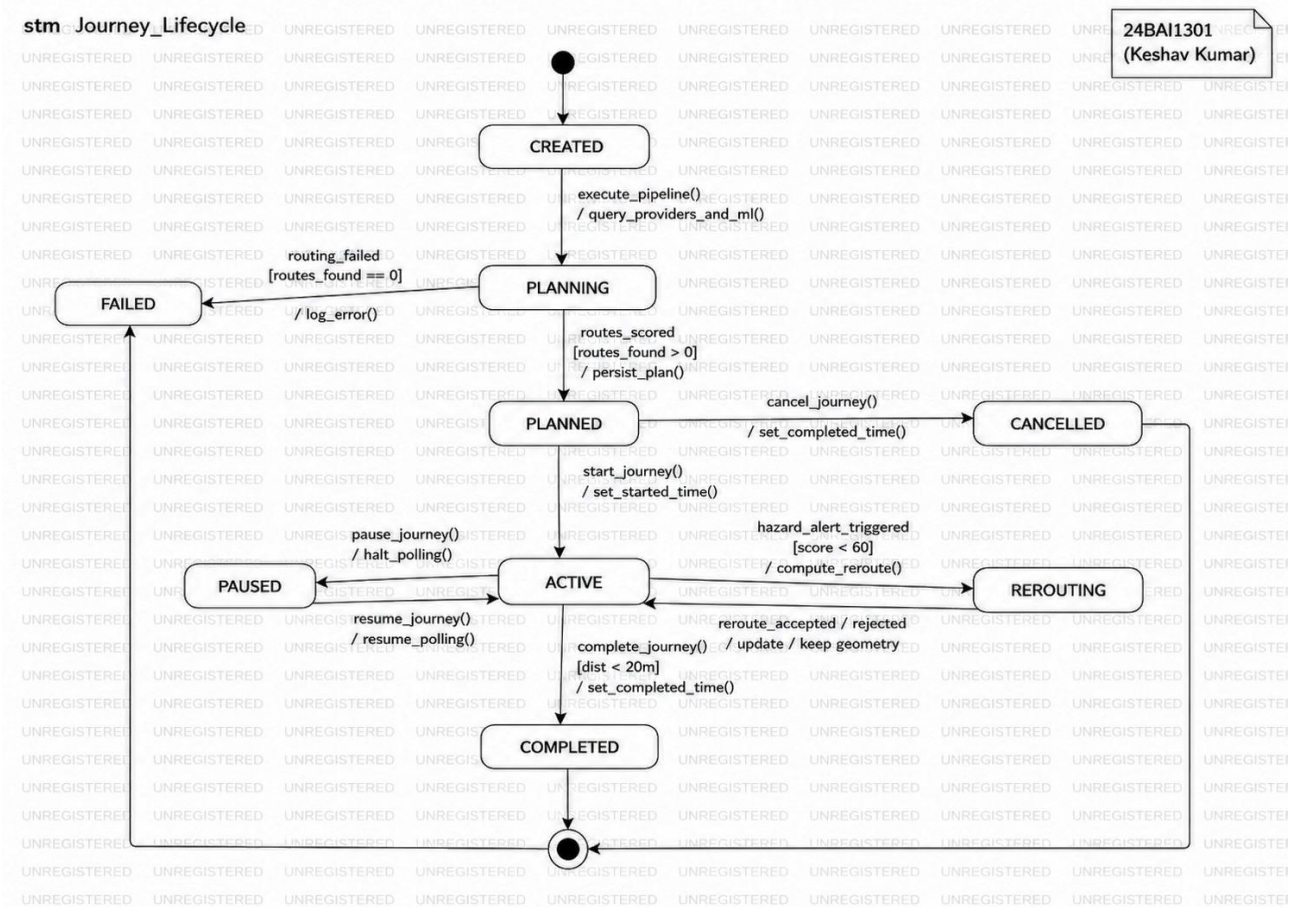

## 6.5 State Machine Diagram

**Purpose:** Formally models the lifecycle states, trigger events, guard conditions, and actions of the Journey domain aggregate.

**Key Elements — States:** CREATED → PLANNING → PLANNED → ACTIVE → {PAUSED, REROUTING} → {COMPLETED, CANCELLED, FAILED}.

| Current State | Trigger (Guard) | Action | Next State |
|---|---|---|---|
| CREATED | execute_pipeline() | query_providers_and_ml() | PLANNING |
| PLANNING | routes_scored (routes_found>0) | persist_plan() | PLANNED |
| PLANNING | routing_failed (routes_found==0) | log_error() | FAILED |
| PLANNED | start_journey() | set_started_time() | ACTIVE |
| PLANNED | cancel_journey() | set_completed_time() | CANCELLED |
| ACTIVE | pause_journey() | halt_polling() | PAUSED |
| PAUSED | resume_journey() | resume_polling() | ACTIVE |
| ACTIVE | hazard_alert_triggered (score<60) | compute_reroute() | REROUTING |
| REROUTING | reroute_accepted / rejected | update / keep geometry | ACTIVE |
| ACTIVE | complete_journey() (dist<20m) | set_completed_time() | COMPLETED |

**Brief Description:** Primary path: CREATED → PLANNING → PLANNED → ACTIVE → COMPLETED. ACTIVE loops to PAUSED and REROUTING (triggered when safety_score < 60), returning to ACTIVE. CANCELLED is reachable from PLANNED or ACTIVE; FAILED only from PLANNING when no routes are found.




## 6.6 Component Diagram

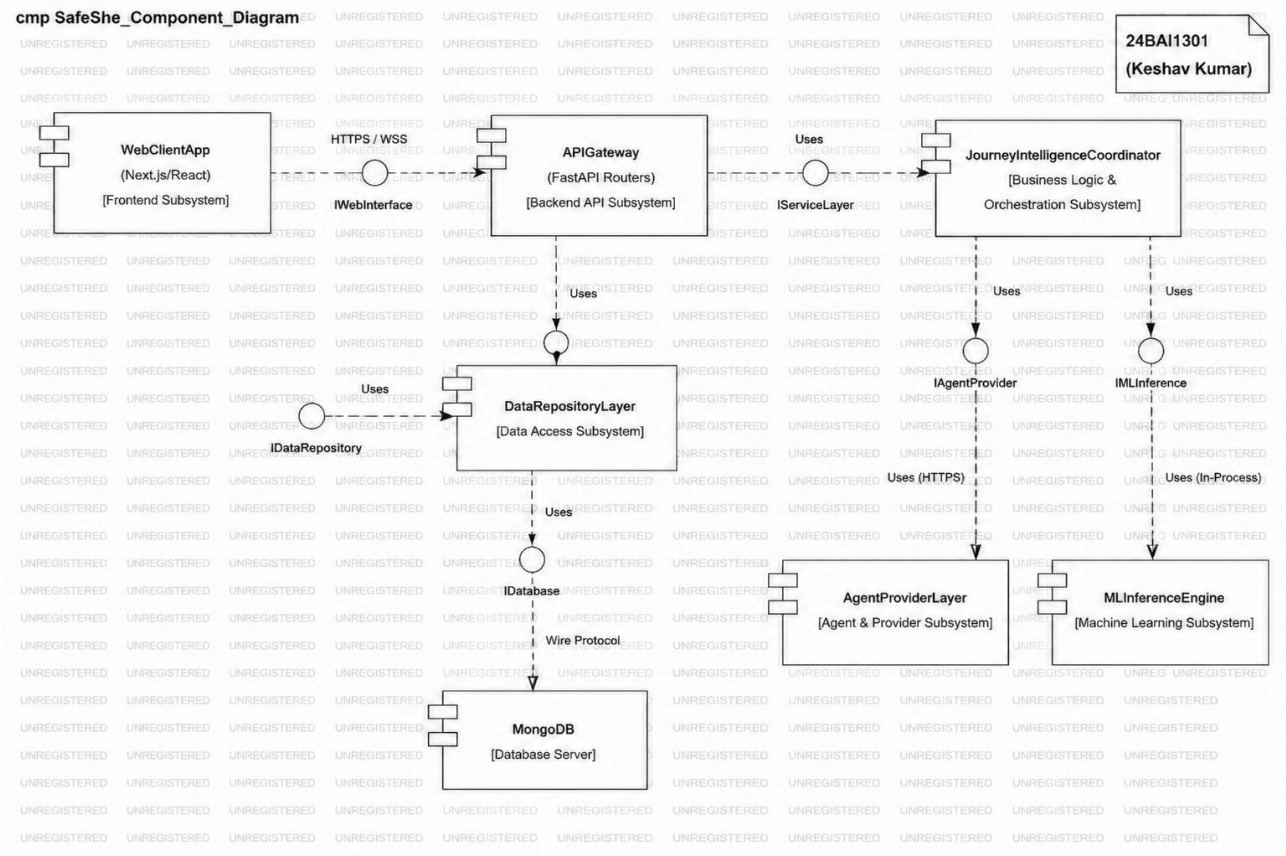
**Purpose:** Illustrates the physical modular software components, provided/required interfaces, and structural dependencies.

**Key Elements — Components:**

- WebClientApp (Next.js/React) — Frontend Subsystem.
- APIGateway (FastAPI Routers) — Backend API Subsystem.

- JourneyIntelligenceCoordinator — Business Logic & Orchestration Subsystem.
- AgentProviderLayer — Agent & Provider Subsystem.
- MLInferenceEngine — Machine Learning Subsystem.
- DataRepositoryLayer — Data Access Subsystem.

**Brief Description:** WebClientApp connects to APIGateway over HTTPS/WSS; APIGateway calls the Service Layer, which delegates to JourneyIntelligenceCoordinator and reads/writes via DataRepositoryLayer. The Coordinator fans out to AgentProviderLayer (external HTTPS calls) and MLInferenceEngine (in-process inference); DataRepositoryLayer persists to MongoDB over the wire protocol.




## 6.7 Deployment Diagram

**Purpose:** Specifies the physical nodes, execution environments, artifacts, ports, and network protocols used to deploy SafeShe.

**Key Elements — Nodes:**

- Client User Device (browser) — runs the Next.js SPA bundle.
- Application Server (Linux/Docker, Uvicorn/Python 3.10) — runs the FastAPI backend and in-memory XGBoost model.
- Database Server — MongoDB 7.0 Community with users, journeys, community_reports (2dsphere) collections.
- External Cloud APIs — OSRM, Nominatim, OpenWeatherMap, Twilio.

**Brief Description:** The client connects to the Application Server via HTTPS/WSS (Port 443, TLS 1.3). The Application Server persists to the Database Server via the MongoDB wire protocol (Port 27017) and issues outbound HTTPS calls to the external cloud APIs as needed to fulfil each request.

# 7. Data Flow Modeling

## 7.1 Level 0 DFD (Context Diagram)

The Context Diagram treats SafeShe as a single process bounded by the actors and external systems that exchange data with it.

**External Entities:** Commuter, Emergency Contact, System Administrator, OSRM Routing Service, Nominatim Geocoding Service, OpenWeatherMap Service, Twilio SMS Gateway, LLM Provider.

| Flow | Data Exchanged |
|---|---|
| Commuter ↔ System | Registration/login, journey requests, GPS telemetry, SOS trigger, hazard reports, chat queries ↔ route plans, safety scores, alerts, chat responses, dashboard data |
| System ↔ Emergency Contact | SOS SMS alert with live-tracking link; live GPS broadcast |
| System ↔ OSRM Service | Route request (origin, destination, mode) ↔ route geometries and distances |
| System ↔ Nominatim Service | Address/reverse-geocode query ↔ resolved coordinates/address |
| System ↔ OpenWeatherMap | Location query ↔ weather data (precipitation, visibility) |
| System ↔ Twilio Gateway | SMS dispatch request ↔ delivery status |
| System ↔ LLM Provider | Explanation/chat prompt ↔ generated explanation/chat response |
| System ↔ Administrator | Telemetry logs, moderation queue ↔ moderation decisions |

*[INSERT DFD LEVEL 0 HERE]*

## 7.2 Level 1 DFD

Level 1 decomposes the single Level 0 process bubble into the system's principal processes and data stores.

**Data Stores:** D1 users · D2 journeys · D3 community_reports · D4 chat_sessions · D5 emergency_sessions (in-memory).

| Process | Name | Input Flow | Store(s) Touched |
|---|---|---|---|
| P1 | User Account Management | Registration/login, preference updates | D1 |
| P2 | Safe Journey Planning & Scoring | Origin, destination, transport mode | D1, D2 |
| P3 | Live Journey Monitoring & Rerouting | GPS telemetry stream | D2, D3 |
| P4 | Emergency SOS Orchestration | SOS trigger, live GPS packets | D2, D5 |
| P5 | Community Hazard Reporting | CommunityReportCreate DTO | D3 |
| P6 | Conversational AI Assistant | ChatRequest DTO | D4 |
| P7 | City Safety Dashboard Aggregation | user_id, current coordinates | D2, D3 |
| P8 | Administrative Audit & Moderation | Moderation decisions, audit queries | D2, D3 |

*[INSERT DFD LEVEL 1 HERE]*

# 8. Risk Analysis

## 8.1 Risk Identification

| ID | Category | Description |
|---|---|---|
| R1 | Technical | Third-party API unavailability (OSRM, Nominatim, OpenWeatherMap) degrades route quality or blocks planning. |

| ID | Category | Description |
|---|---|---|
| R2 | Technical | ML safety-score model bias/inaccuracy from synthetic training proxies produces misleading scores. |
| R3 | Security | JWT compromise or IDOR exposes another user's journey or emergency data. |
| R4 | Operational | SMS gateway failure/cost overrun delays emergency-contact notification during an active SOS. |
| R5 | Performance | WebSocket pool exceeds capacity during a mass emergency event, degrading broadcast latency. |
| R6 | Data/Privacy | Improper retention or leakage of GPS trails and emergency session logs. |
| R7 | Schedule | Parallel Frontend/Backend/ML streams slip and delay the Integration milestone. |
| R8 | Domain | False or malicious community hazard reports degrade trust in the crowdsourced layer. |
| R9 | Environmental | GPS inaccuracy in dense urban areas reduces reroute and hazard-proximity precision. |
| R10 | Compliance | Non-conformance with WCAG 2.1 AA reduces usability in a safety-critical context. |

## 8.2 Risk Assessment Matrix (Probability vs. Impact)

| ID | Probability | Impact | Risk Level |
|---|---|---|---|
| R1 | High | Medium | High |
| R2 | Medium | High | High |
| R3 | Low | Critical | High |
| R4 | Medium | Critical | Critical |
| R5 | Low | High | Medium |
| R6 | Low | Critical | High |
| R7 | Medium | Medium | Medium |
| R8 | Medium | Medium | Medium |
| R9 | High | Low | Medium |
| R10 | Medium | Medium | Medium |

Risk Level = Probability × Impact (Low/Medium/High/Critical). R4 is rated Critical because it combines plausible probability with life-safety impact; R3 and R6 are rated High despite low probability because their impact on user safety and privacy is severe.

## 8.3 Risk Mitigation Strategies

| ID | Mitigation Strategy |
|---|---|
| R1 | Circuit-breaking / graceful fallback (NFR-005): straight-line geometry or cached weather on timeout (>5000 ms). |
| R2 | Safety score treated as explanatory input, not an opaque verdict (FR-013); domain score cap (DR-002) overrides ML output for verified severe hazards. |
| R3 | TLS 1.3, BCrypt (factor 12), HMAC-SHA256 JWTs (NFR-003); mandatory owner-ID validation on every endpoint (NFR-004). |
| R4 | Fail-open emergency tracking (DR-003): WebSocket broadcast continues independent of SMS; failed SMS queued for retry. |
| R5 | 10,000 concurrent-connection target via async I/O and stateless broadcast pools (NFR-007), enabling horizontal scaling. |
| R6 | Data retention by sensitivity class (DR-004): GPS trails generalised after 30 days; emergency logs bounded to 90 days. |

| ID | Mitigation Strategy |
|---|---|
| R7 | Explicit WBS parallelisation of Frontend/Backend/ML streams with one tracked Integration milestone and dependency-aware predecessors. |
| R8 | Community trust threshold (DR-006), duplicate-report suppression (FR-011), and administrator moderation (UC-14). |
| R9 | 100 m route-deviation buffer before reroute is triggered (FR-008), absorbing normal GPS jitter. |
| R10 | WCAG 2.1 AA mandated as an explicit, testable requirement (NFR-009), not a best-effort goal. |

# 9. Testing Strategy

## 9.1 Test Plan

- **Scope:** All 15 Functional Requirements and 11 NFRs (Chapter 4) are in scope for verification.
- **Approach:** Four-tier strategy in strict order — Unit, Integration, System/End-to-End, Acceptance — each tier gated on the prior tier passing; every test case is cross-referenced to an FR in the RTM (Appendix C).
- **Test Environment:** Unit/Integration tests run against an isolated test MongoDB with mocked external providers; System/Acceptance tests run against staging with sandbox Twilio credentials.
- **Entry/Exit Criteria:** A module's own unit tests must pass before Integration begins, and Integration before System begins. Exit requires all test cases passing, full RTM coverage, and no open Critical/High-severity defect.

## 9.2 Unit Testing

| Test ID | Req. | Steps | Expected Result |
|---|---|---|---|
| TC-UNIT-001 | FR-001 | verify_password("Password123!") on a hashed instance | Returns True |
| TC-UNIT-002 | FR-003 | update_preferences(prefs) with normalized payload | Preferences updated in place |
| TC-UNIT-003 | FR-006 | transition_to(PLANNING) from CREATED state | State → PLANNING; history updated |
| TC-UNIT-004 | FR-011 | normalize(report_dict) on raw community report input | Flattened, typed dict with required keys |

## 9.3 Integration Testing

| Test ID | Req. | Steps | Expected Result |
|---|---|---|---|
| TC-INT-001 | FR-001 | POST /users/ with unique credentials | HTTP 201; user persisted |
| TC-INT-002 | FR-002 | POST /users/login with valid credentials | HTTP 200; access_token returned |
| TC-INT-003 | FR-004 | POST /users/me/emergency-contacts | HTTP 201; 6th contact rejected |
| TC-INT-004 | FR-005 | POST /journeys/ with source/destination, mode=WALK | HTTP 200; 1–3 scored routes |
| TC-INT-005 | FR-006 | POST /journeys/{id}/start on PLANNED journey | HTTP 200; state=ACTIVE |
| TC-INT-006 | FR-009 | POST /emergency/sos with location | HTTP 200; session DISPATCHED |
| TC-INT-007 | FR-010 | Open WS /ws/emergency/{id}; send GPS packet | Server broadcasts to observers |

| Test ID | Req. | Steps | Expected Result |
|---|---|---|---|
| TC-INT-008 | FR-011 | POST /community/ with location, type, severity | HTTP 201; duplicate within 50 m/1h rejected |
| TC-INT-009 | FR-012 | GET /community/nearby?radius=5000 | HTTP 200; results within radius |
| TC-INT-010 | FR-014 | GET /dashboard/overview | HTTP 200; city_safety_score returned |
| TC-INT-011 | FR-015 | GET /journeys/history | HTTP 200; results scoped to user |

## 9.4 System Testing

| Test ID | Scenario | Steps | Expected Result |
|---|---|---|---|
| TC-SYS-001 | End-to-End Safe Navigation | Plan → Select → Start → Complete | State: CREATED → PLANNED → ACTIVE → COMPLETED |
| TC-SYS-002 | Live Telemetry (FR-007) | Start journey; stream GPS for 5 min | Continuous distance, ETA, score trend |
| TC-SYS-003 | Dynamic Reroute (FR-008) | Inject hazard on active route | ReroutePlan generated with risk-delta explanation |
| TC-SYS-004 | Emergency SOS E2E (FR-009) | Trigger SOS; verify SMS + WS | SMS within 300 ms; observers get live GPS |
| TC-SYS-005 | AI Assistant Query (FR-013) | Chat: "nearest police station" | Response includes safe_pois list |

### *9.4.1 Security & Performance*

| Test ID | Req. | Steps | Expected Result |
|---|---|---|---|
| TC-SEC-001 | NFR-003 | Repeated failed login attempts | HTTP 401 each; rate limiter engages beyond 10/min |
| TC-PERF-001 | NFR-007 | 10,000 concurrent WebSocket connections | All connections stable; latency within target |
| TC-PERF-002 | NFR-008 | Radial hazard query on 1,000,000 seeded records | Completes under 20 ms |

## 9.5 Acceptance Testing

| Test ID | User Story | Scenario | Pass Criteria |
|---|---|---|---|
| TC-ACC-001 | Safe Night Walking (FR-005) | Nighttime request for a walking route | Suggests well-lit routes; explanation cites lighting score |
| TC-ACC-002 | Instant Hazard Reporting (FR-011) | User drops a pin for a broken streetlight | Pin appears on the community map within 1 second |

# 10. Glossary

| Term | Definition |
|---|---|
| Ambient Risk Level | City/area-wide qualitative safety indicator from active hazard reports, weather, and |

| Term | Definition |
|---|---|
| | time-of-day. |
| BCrypt | Password-hashing algorithm used to store credentials irreversibly. |
| Community Report / Hazard Pin | Crowdsourced, geolocated record of a safety concern submitted by a Commuter. |
| Commuter | Primary End User of SafeShe who plans and undertakes journeys. |
| DTO (Data Transfer Object) | Structured object carrying data between client and API layer. |
| Emergency Contact | Person nominated to receive SOS alerts and monitor live tracking. |
| E.164 | International telephone-number formatting standard used for SMS dispatch. |
| Feature Tensor | 5-dimensional numeric vector engineered from route/weather/hazard data for the ML model. |
| Geocoding | Converting a human-readable address into geographic coordinates, and vice versa. |
| GeoJSON | JSON-based format encoding geographic points, lines, and polygons. |
| Haversine Formula | Formula for computing great-circle distance between two GPS coordinates. |
| IDOR | Insecure Direct Object Reference — accessing another user's resource via a guessed/manipulated identifier. |
| Journey | Core domain aggregate representing a single planned or active trip. |
| Journey Intelligence Coordinator (JIC) | Internal orchestrator coordinating routing, weather, ML scoring, and LLM explanation. |
| JWT (JSON Web Token) | Signed token used to authenticate a user across API requests. |
| LLM | Large Language Model — synthesizes natural-language route explanations and assistant responses. |
| POI (Point of Interest) | Named location such as a police station, hospital, or pharmacy. |
| Reroute | System-initiated proposal to switch an active journey to a safer alternative route. |
| Safety Score | ML-derived numeric score in [0,100] indicating relative route safety. |
| SOS Session | Active emergency-distress record created when a Commuter triggers SOS. |
| TTL Index | MongoDB index that automatically expires documents after a configured period. |
| 2dsphere Index | MongoDB index type enabling efficient geospatial queries. |
| Verification Status | Moderation state of a community report (PENDING, VERIFIED, DISMISSED). |
| WCAG | Web Content Accessibility Guidelines — accessibility standard SafeShe's UI conforms to. |
| WebSocket | Persistent bidirectional protocol (RFC 6455) used for real-time telemetry and emergency broadcast. |
| XGBoost | Gradient-boosted decision tree algorithm used as the safety-scoring ML model. |

# 11. Appendices

## Appendix A — References

- IEEE Std 830-1998 — IEEE Recommended Practice for Software Requirements Specifications.
- OMG Unified Modeling Language (OMG UML) Superstructure, Version 2.5.1.
- RFC 7519 — JSON Web Token (JWT) Architecture and Validation.
- RFC 6455 — The WebSocket Protocol. RFC 7946 — The GeoJSON Format.
- W3C Web Content Accessibility Guidelines (WCAG) 2.1.

- MongoDB Manual — Geospatial Queries and 2dsphere Indexes.
- FastAPI Documentation (fastapi.tiangolo.com); OSRM API Documentation (project-osrm.org).
- StarUML User Guide; ProjectLibre User Guide.

## Appendix B — Entity–Relationship (ER) Diagram & Data Dictionary

Although SafeShe persists data in MongoDB (a document database), the collections below are modeled with explicit relationships and cardinalities for ER diagramming purposes.

**Entities:** users, journeys, community_reports, chat_sessions.

| Entity | Related Entity | Cardinality | Relationship |
|---|---|---|---|
| users | journeys | 1 : M | One user undertakes many journeys |
| journeys | community_reports | 0..* : 0..* | Route may reference nearby hazard pins (resolved by geospatial query, not a stored FK) |
| users | community_reports | 1 : M | One user submits many hazard reports |
| users | chat_sessions | 1 : M | One user has many chat sessions |
| journeys | chat_sessions | 1 : 0..* | A journey may be linked to zero or more chat sessions |

**Attributes — users:** _id (PK), username, email (unique), password_hash (BCrypt), preferences (embedded), emergency_contacts (array, max 5), created_at, is_active.

**Attributes — journeys:** _id (PK), user_id (FK), source/destination (GeoJSON Point), plan (embedded), state (embedded, indexed), progress (embedded), alerts (array), created_timestamp.

**Attributes — community_reports:** _id (PK), user_id (FK, nullable), location (GeoJSON, 2dsphere), report_type (enum), severity (enum), description, verification_status, upvotes/downvotes, created_at (TTL 7 days).

Normalization: all collections satisfy 1NF–BCNF — scalar fields are atomic, repeated groups are typed sub-document arrays, and every non-key attribute is fully functionally dependent on the primary key.

*[INSERT ER DIAGRAM HERE]*

## Appendix C — Requirement Traceability Matrix (RTM)

| Req. | Module | API | Database | UML Diagram | Test Case(s) |
|---|---|---|---|---|---|
| FR-001 | UserService | POST /users/ | users | Class, Use Case, Sequence | TC-UNIT-001, TC-INT-001 |
| FR-002 | AuthService | POST /users/login | users | Class, Use Case, Sequence | TC-INT-002, TC-SEC-001 |
| FR-003 | UserService | PATCH /users/me/preferences | users.preferences | Class, Use Case | TC-UNIT-002 |
| FR-004 | UserService | POST /users/me/emergency-contacts | users.emergency_contacts | Class, State | TC-INT-003 |
| FR-005 | JourneyIntelligence | POST /journeys/ | journeys.plan | Class, Use Case, Sequence, Activity | TC-INT-004, TC-SYS-001 |
| FR-006 | JourneyService | POST /journeys/{id}/start | journeys.state | Class, State, Activity | TC-UNIT-003, TC-INT-005 |

| Req. | Module | API | Database | UML Diagram | Test Case(s) |
|---|---|---|---|---|---|
| FR-007 | LiveMonitorEngine | GET /journeys/{id}/monitor | journeys.progress | Class, Sequence | TC-SYS-002 |
| FR-008 | JourneyCoordinator | POST /journeys/{id}/reroute | journeys.plan | Class, State, Activity | TC-SYS-003 |
| FR-009 | EmergencyService | POST /emergency/sos | emergency_sessions | Class, Use Case, Sequence | TC-INT-006, TC-SYS-004 |
| FR-010 | WebSocketManager | WS /ws/emergency/{id} | In-Memory WS Pool | Class, Sequence | TC-INT-007, TC-PERF-001 |
| FR-011 | CommunityService | POST /community/ | community_reports | Class, Use Case, ER | TC-UNIT-004, TC-INT-008 |
| FR-012 | CommunityService | GET /community/nearby | community_reports | Class, Component, ER | TC-INT-009, TC-PERF-002 |
| FR-013 | AssistantService | POST /assistant/chat | chat_sessions | Class, Component, Sequence | TC-SYS-005 |
| FR-014 | DashboardAgent | GET /dashboard/overview | Aggregation Pipeline | Class, Component | TC-INT-010 |
| FR-015 | JourneyService | GET /journeys/history | journeys | Class, Repository | TC-INT-011 |

## Appendix D — REST & WebSocket API Catalogue

| Method | Endpoint | Summary | Auth |
|---|---|---|---|
| POST | /users/ | Register a new user account | None |
| POST | /users/login | Authenticate and issue a JWT | None |
| PATCH | /users/me/preferences | Update transport mode & notification preferences | Bearer JWT |
| POST | /users/me/emergency-contacts | Add an emergency contact (max 5) | Bearer JWT |
| POST | /journeys/ | Plan a safe multimodal route | Bearer JWT |
| POST | /journeys/{id}/start | Transition a journey to ACTIVE | Bearer JWT |
| POST | /journeys/{id}/pause \| /resume \| /complete | Pause / resume / complete a journey | Bearer JWT |
| POST | /journeys/{id}/reroute | Request a dynamic reroute | Bearer JWT |
| GET | /journeys/{id}/monitor | Poll live telemetry | Bearer JWT |
| GET | /journeys/history | Retrieve paginated historical journeys | Bearer JWT |
| POST | /emergency/sos | Trigger an emergency SOS broadcast | Bearer JWT |
| WS | /ws/emergency/{session_id} | Real-time distress GPS coordinate stream | Session ID |
| WS | /ws/journey/{journey_id} | Active journey telemetry & hazard alerts | Bearer JWT |
| POST | /community/ | Submit a crowdsourced hazard report | Bearer JWT |
| GET | /community/nearby | Retrieve hazard pins within a radius | Bearer JWT |
| POST | /assistant/chat | Query the conversational AI assistant | Bearer JWT |
| GET | /dashboard/overview | Retrieve the city-wide safety dashboard | Bearer JWT |

## Appendix E — Diagram & Submission Checklist

- All seven UML diagrams (Use Case, Activity, Class, Sequence, State Machine, Component, Deployment) drawn in StarUML from Chapter 6, with title blocks (Project Title, Student Name, Registration Number).
- WBS recreated in ProjectLibre from Chapter 5 with all 12 top-level phases, durations, predecessors, and milestone flags; exported as a Gantt chart.
- DFD Level 0 and Level 1 drawn from Chapter 7.
- ER Diagram drawn from Appendix B.
- Requirement Traceability Matrix (Appendix C) cross-checked against the SRS, UML diagrams, and Testing Strategy.
- All [INSERT ... HERE] placeholders replaced with final StarUML / ProjectLibre exports before printing.
- Final Word document compiled, paginated, and reviewed against the faculty rubric.